

NLP

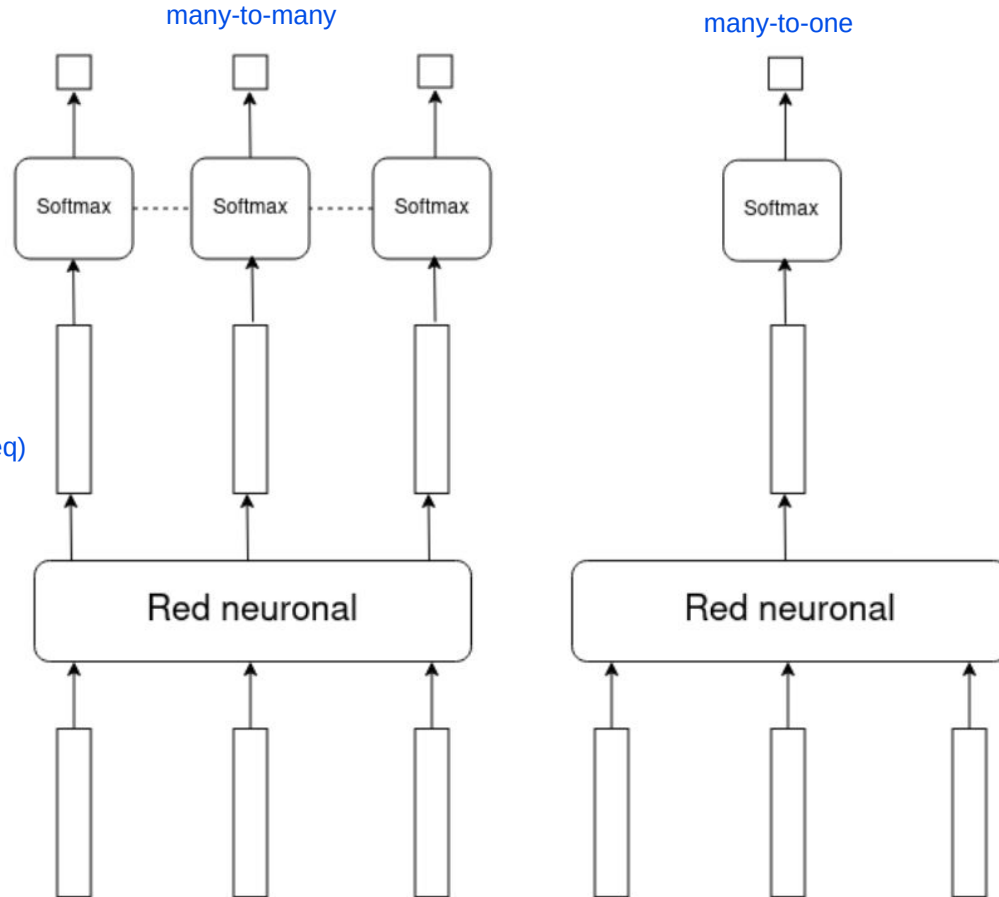
Otras arquitecturas para
procesar secuencias:
CNN y Attention

Josselyn Ordóñez

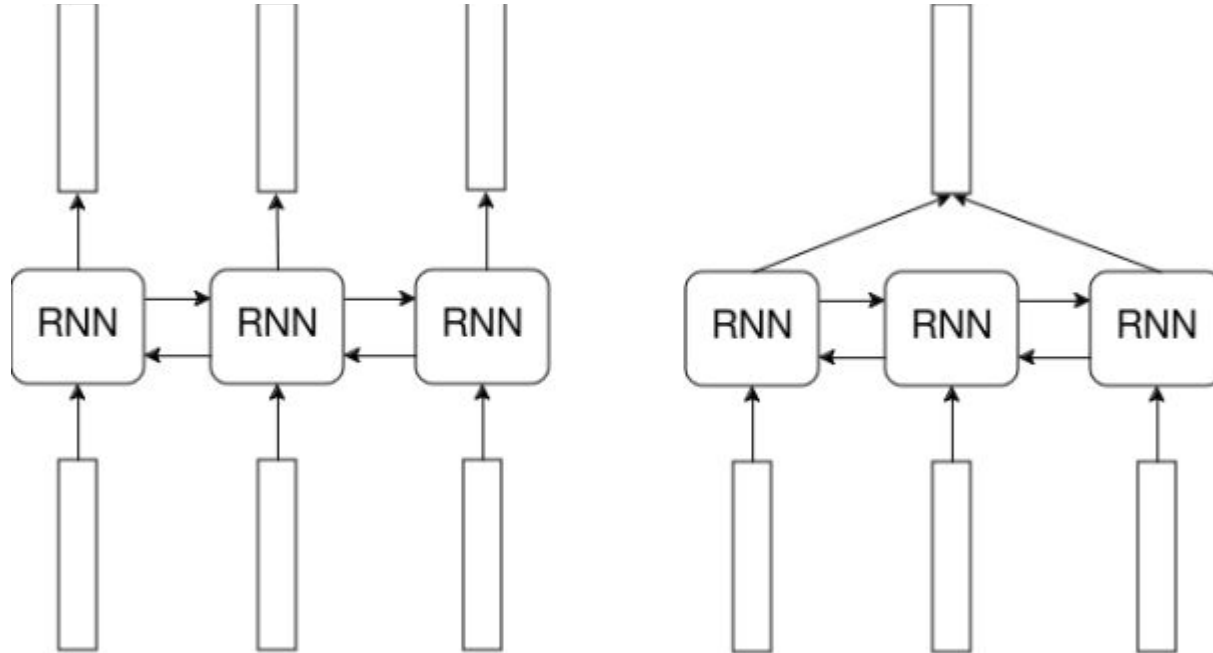
Problemas de clasificación en texto a partir de secuencias



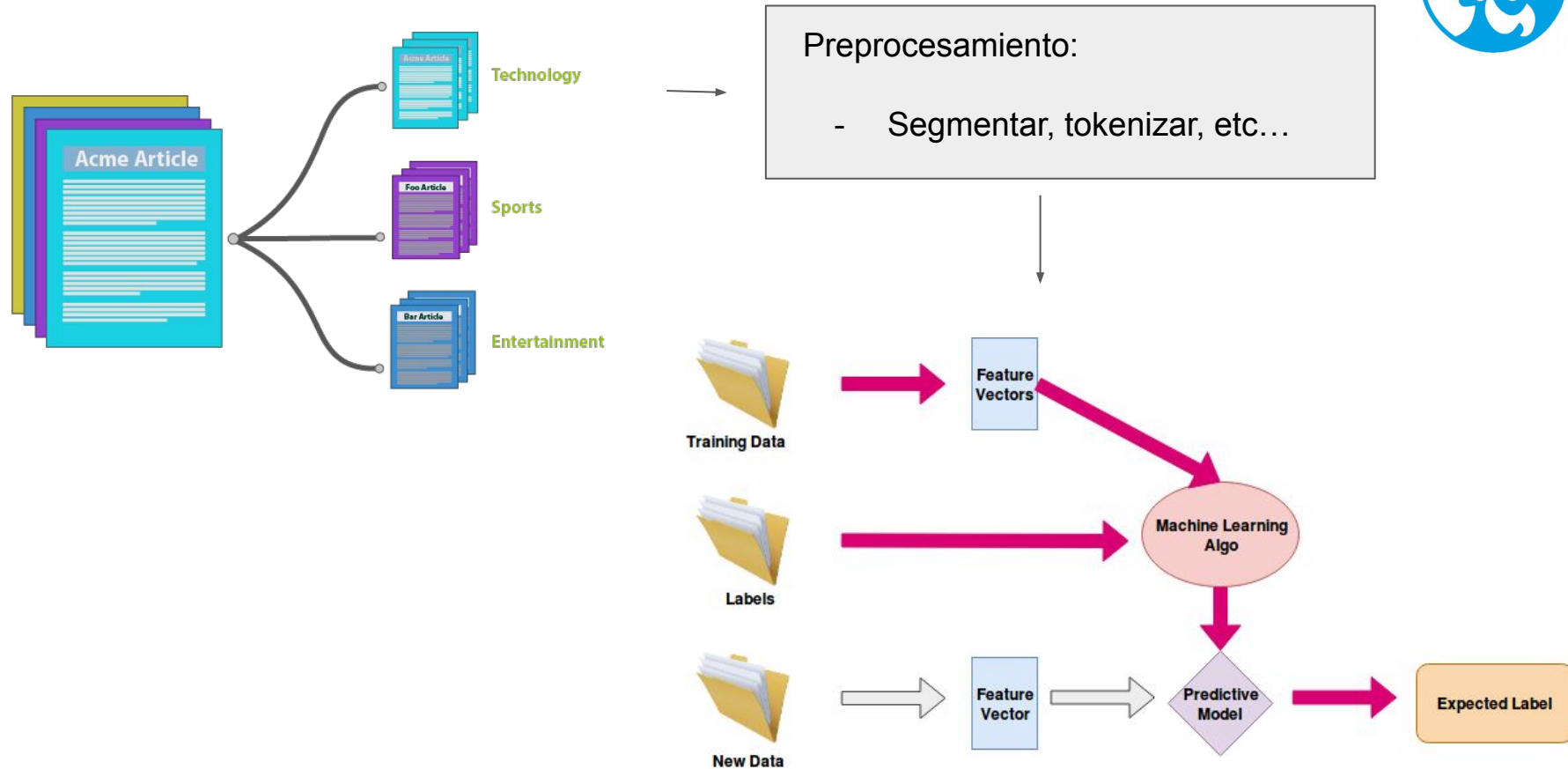
- Clasificación en tópicos many-to-many
- Sentiment analysis
- Modelos de lenguaje (Seq2Seq)
- NER many-to-one
(reconocimiento de entidades/topicos)



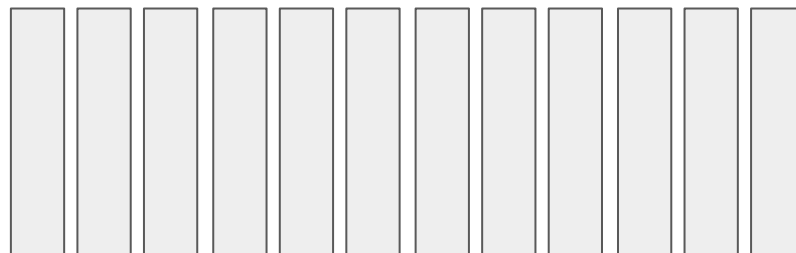
Procesando secuencias con RNNs



Modelo de clasificación de texto



De texto a secuencias de embeddings



A cada identificador se le asigna un embedding el cual captura el significado semantico de los tokens.
Estos embeddings son fijos

Capa de embeddings

[40, 1213, 1215, 20, 578, 11, 120, 79, 31, 26, 608, 974]

Segmentación, normalización, transformación,
filtrado y tokenización PONER MASKING!!

"Cuando Gregorio Samsa se despertó una mañana después de un sueño intranquilo"

Representación One Hot Encoding



Una forma de vectorizar el document es usando OHE, se genera el vocabulario y se pone 1 a la columna donde se encuentra la palabra y 0 en las otras. Y para compararlas se usaba la similitud coseno.

Pero esto puede no ser suficiente ya que no captura el valor semantico. Por lo que es mejor usar Embeddings.

el document es usando OHE,
y se pone 1 a la columna donde
y 0 en las otras. Y para compararlas
seno.

suficiente ya que no captura
o que es mejor

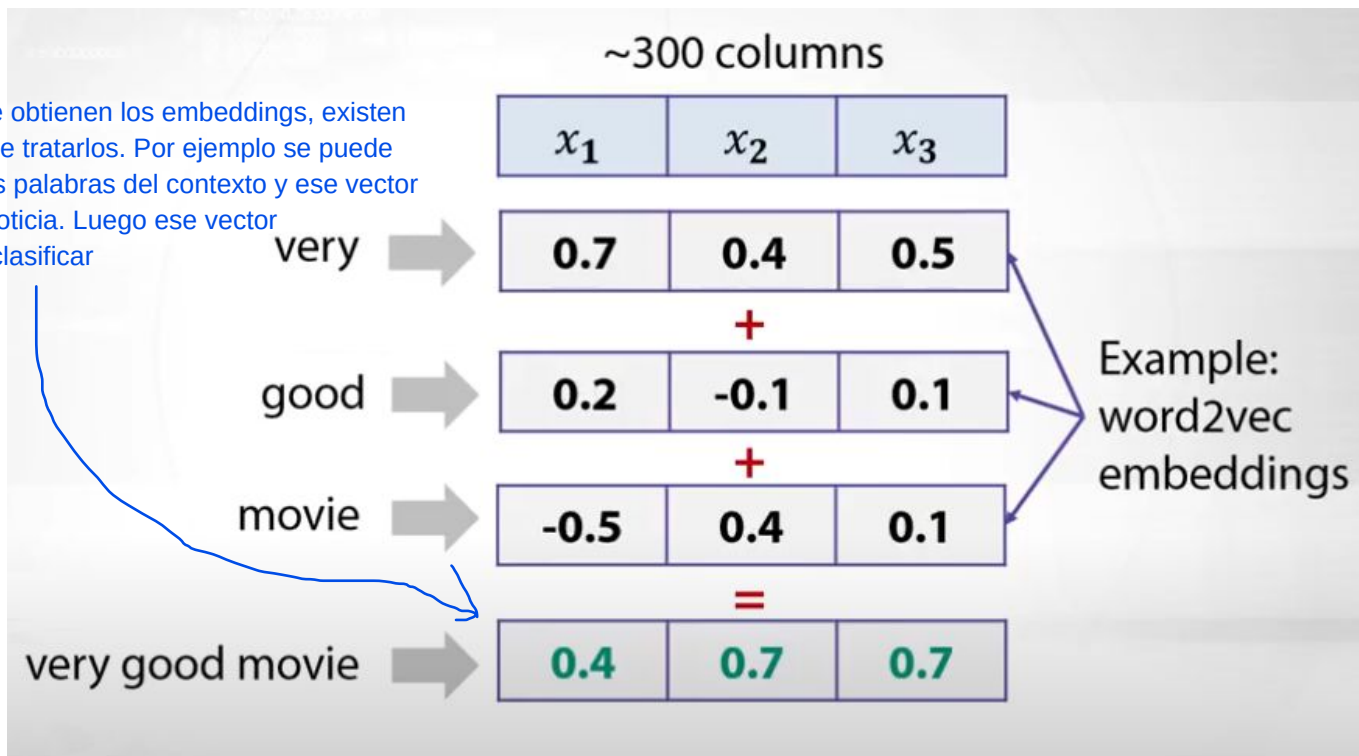
~100k columns

	good	movie	very	a	did	like
very	0	0	1	0	0	0
+						
good	1	0	0	0	0	0
+						
movie	0	1	0	0	0	0
=						
very good movie	1	1	1	0	0	0

Representación por Embedding



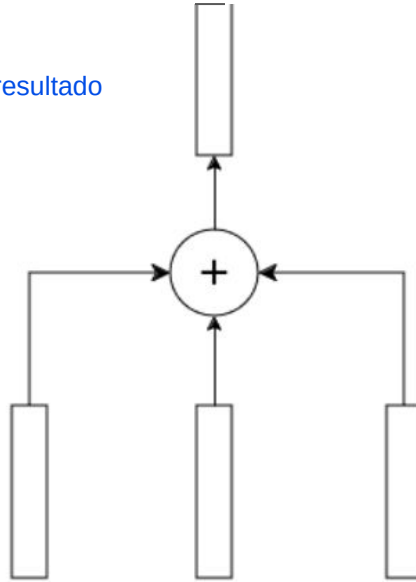
Una vez que se obtienen los embeddings, existen varias formas de tratarlos. Por ejemplo se puede sumar todas las palabras del contexto y ese vector representa la noticia. Luego ese vector se utiliza para clasificar



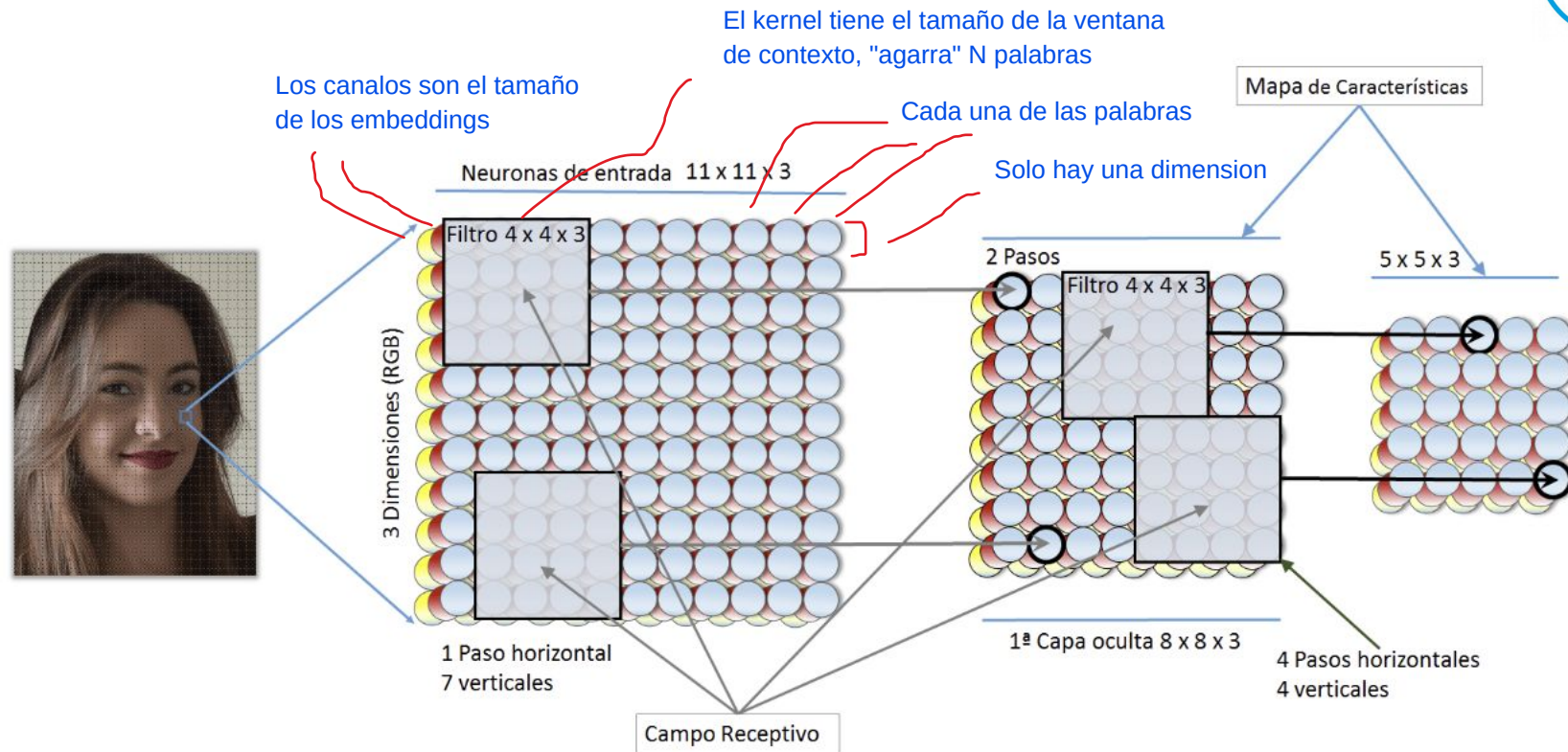
Modelo básico de clasificación a partir de embeddings



Se suman los embeddings y el resultado se utiliza para la clasificación



Arquitecturas Convolucionales

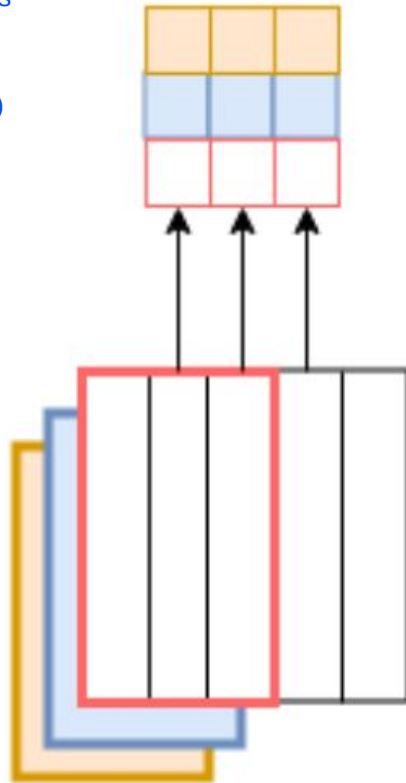


Arquitecturas Convolucionales para procesar secuencias

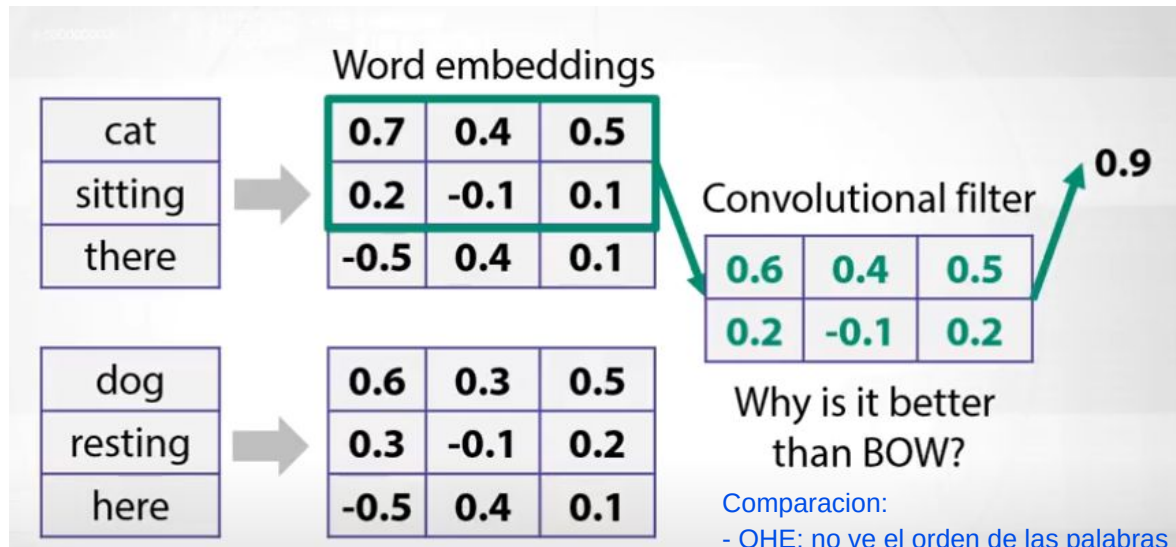


Ahora con texto solo se tiene una dimension, el orden de las palabras. Los canales, ahora es la dimension de los embeddings a usar, y el kernel es unidimensional y de tamaño igual a la ventana de contexto (procesa N palabras)

- Convolución 1D (procesa en secuencia)
- Dimensión de embedding==cantidad de canales
- Tamaño de contexto==tamaño de filtro



Convolución 1D: representación de n-grams



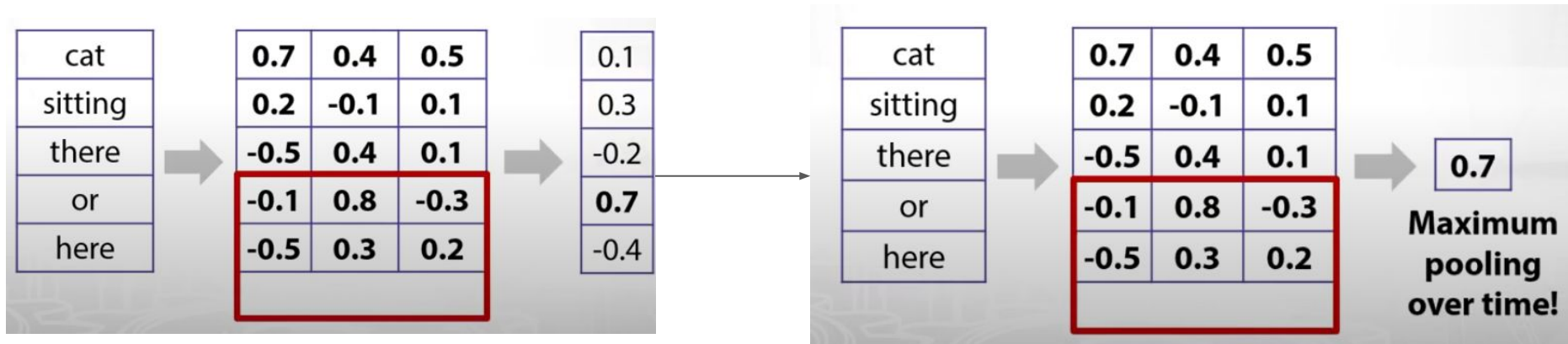
Comparacion:

- OHE: no ve el orden de las palabras ni mantiene el significado semantico
- Kernels: sí procesa en orden las palabras, y no sufre tanto de Vanishing Gradient como en RNN

Pooling: nos interesá qué pero no dónde



El Pooling ayuda a regularizar y principalmente a condensar la información que se está capturando

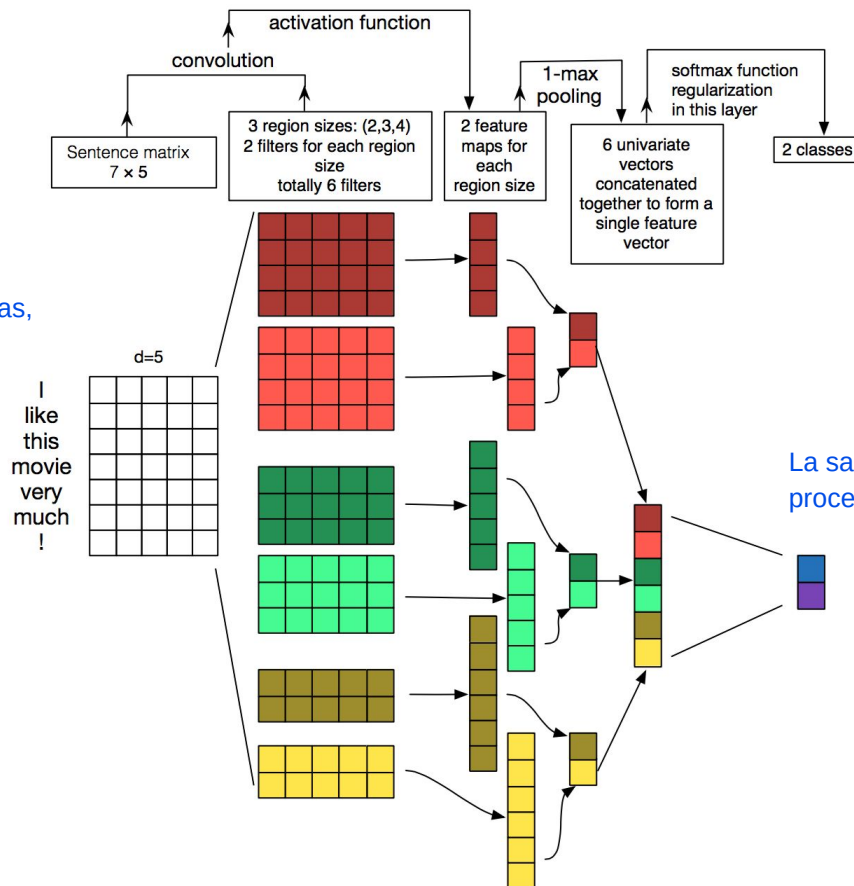


0.7 representa el mejor valor al procesar las palabras.

TextCNN

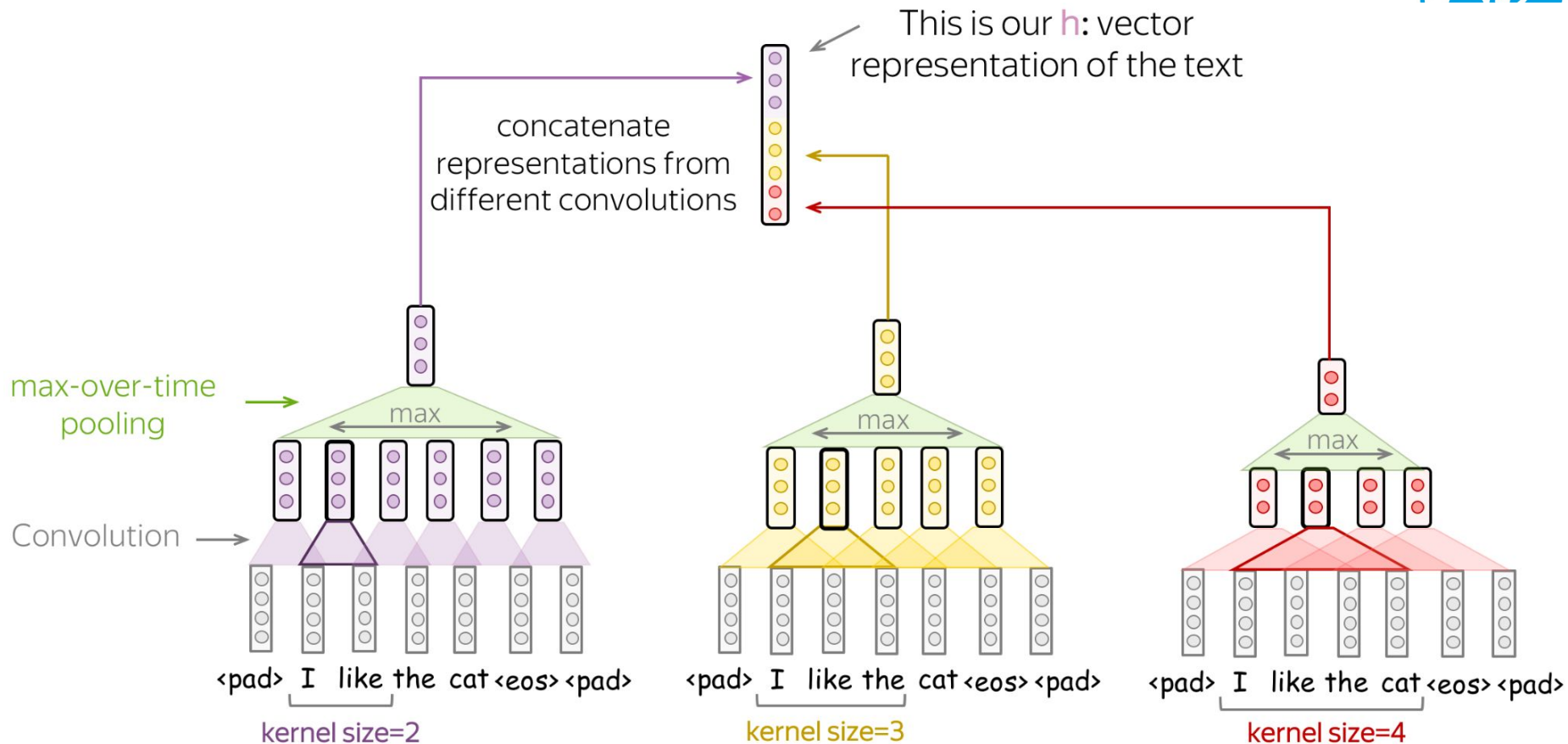


Esta arquitectura entrena diferentes tamaños de kernels para procesar diferentes relaciones entre los tokens. Ejemplo, el kernel de 3 procesa 3 palabras, el kernel de 2 procesa 2 palabras, y así.



La salida condensa la información que se proceso con los diferentes kernels

TextCNN



Mecanismo de atención



En general, un mecanismo de atención es una transformación de secuencias de embeddings a secuencias de embeddings de forma ponderada y paralela.

Ahora con Atención se
puede procesar en paralelo!!
A diferencia de las RNN
que son secuenciales

Existen varios tipos pero las operaciones fundamentales consisten en:

- El cálculo un vector de pesos/scores de atención .
- La construcción de un vector ponderado que es fácilmente paralelizable a lo largo del tamaño de la secuencia.

***¡La degradación del gradiente es independiente del tamaño de la secuencia!
Pero se debe fijar el tamaño máximo de secuencia a procesar.***

*A futuro veremos que el mecanismo de **self-attention** es el que utilizan las arquitecturas **transformer**.*

Feed-forward attention

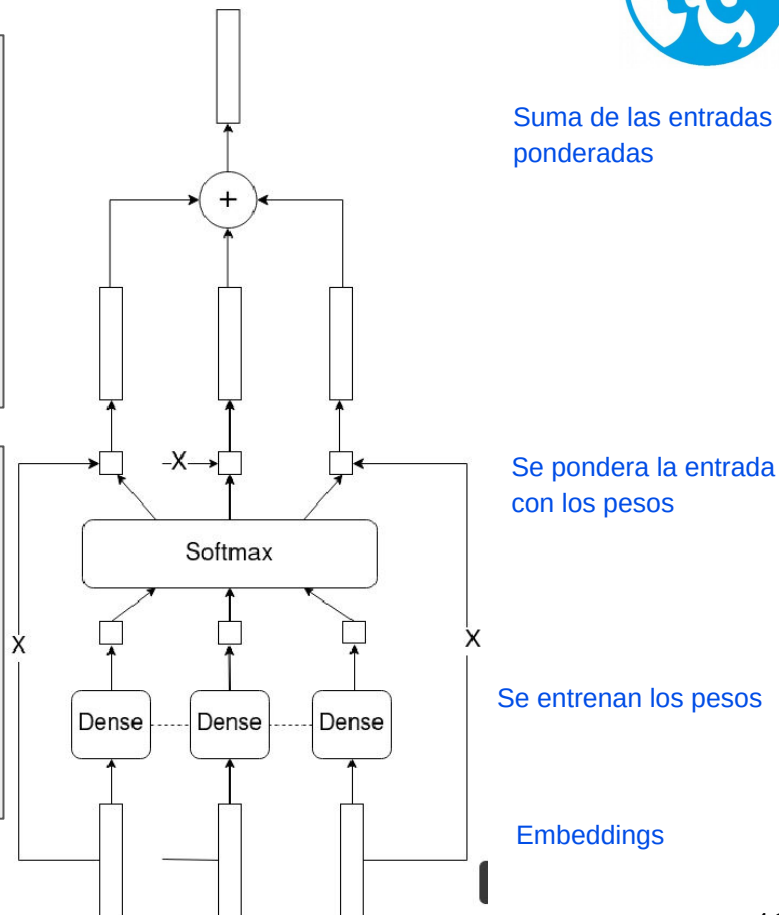
La desventaja de la Atención es que no mantiene el orden las palabras procesadas



Cálculo de vector
ponderado,
paralelizable en el
tamaño de la secuencia

Cálculo de vector de
pesos

¡Es BOW!



Suma de las entradas
ponderadas

Se pondera la entrada
con los pesos

Se entrenan los pesos

Embeddings

Attention via Recurrent Networks

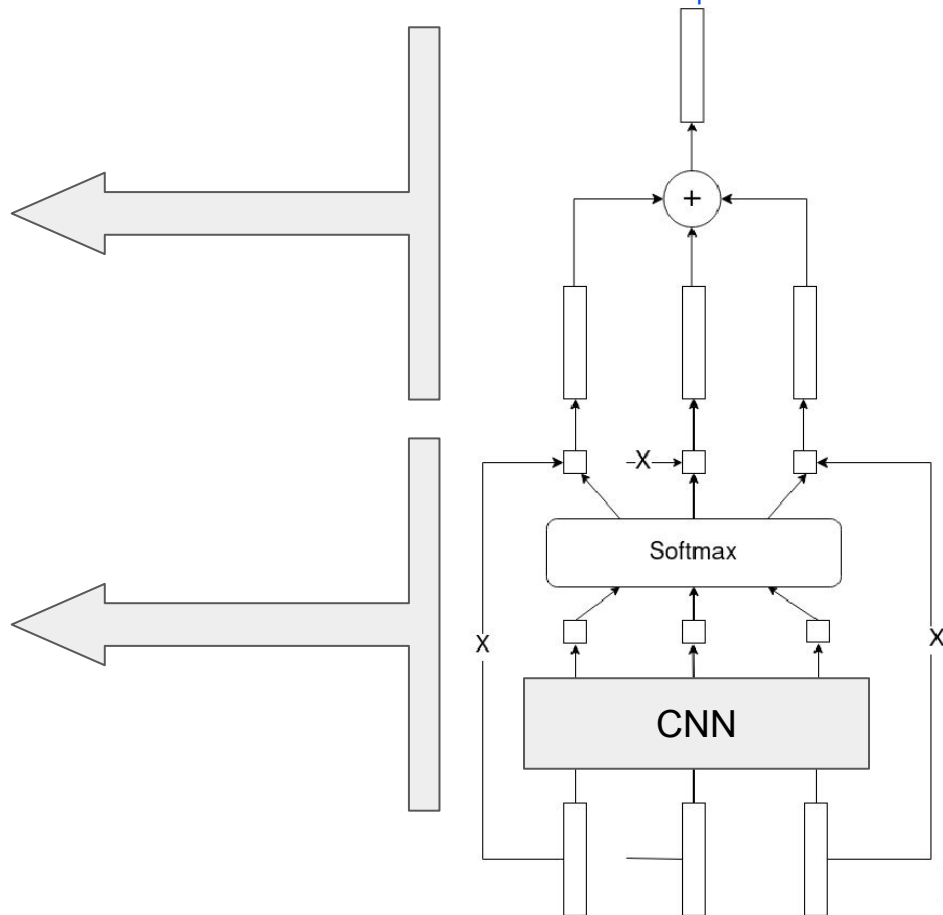
Para procesar las entradas manteniendo el orden se puede usar una capa CNN o RNN



Cálculo de vector ponderado,
paralelizable en el
tamaño de la secuencia

Cálculo de vector de pesos

¡Es recurrente!



Attention via Recurrent Networks



Cálculo de vector
ponderado,
paralelizable en el
tamaño de la secuencia

Cálculo de vector de
pesos

¡Es recurrente!

